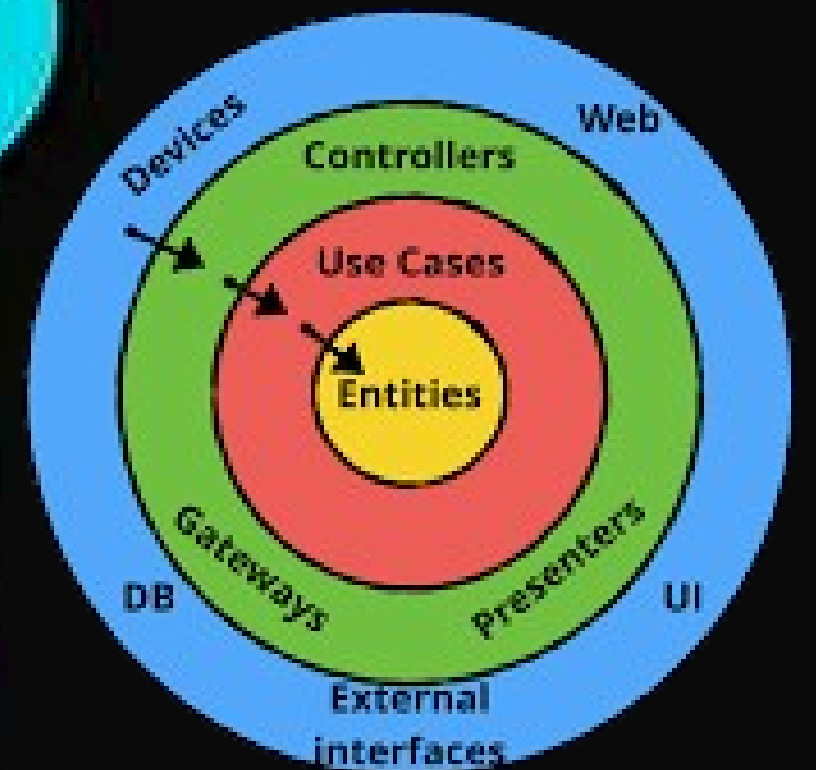
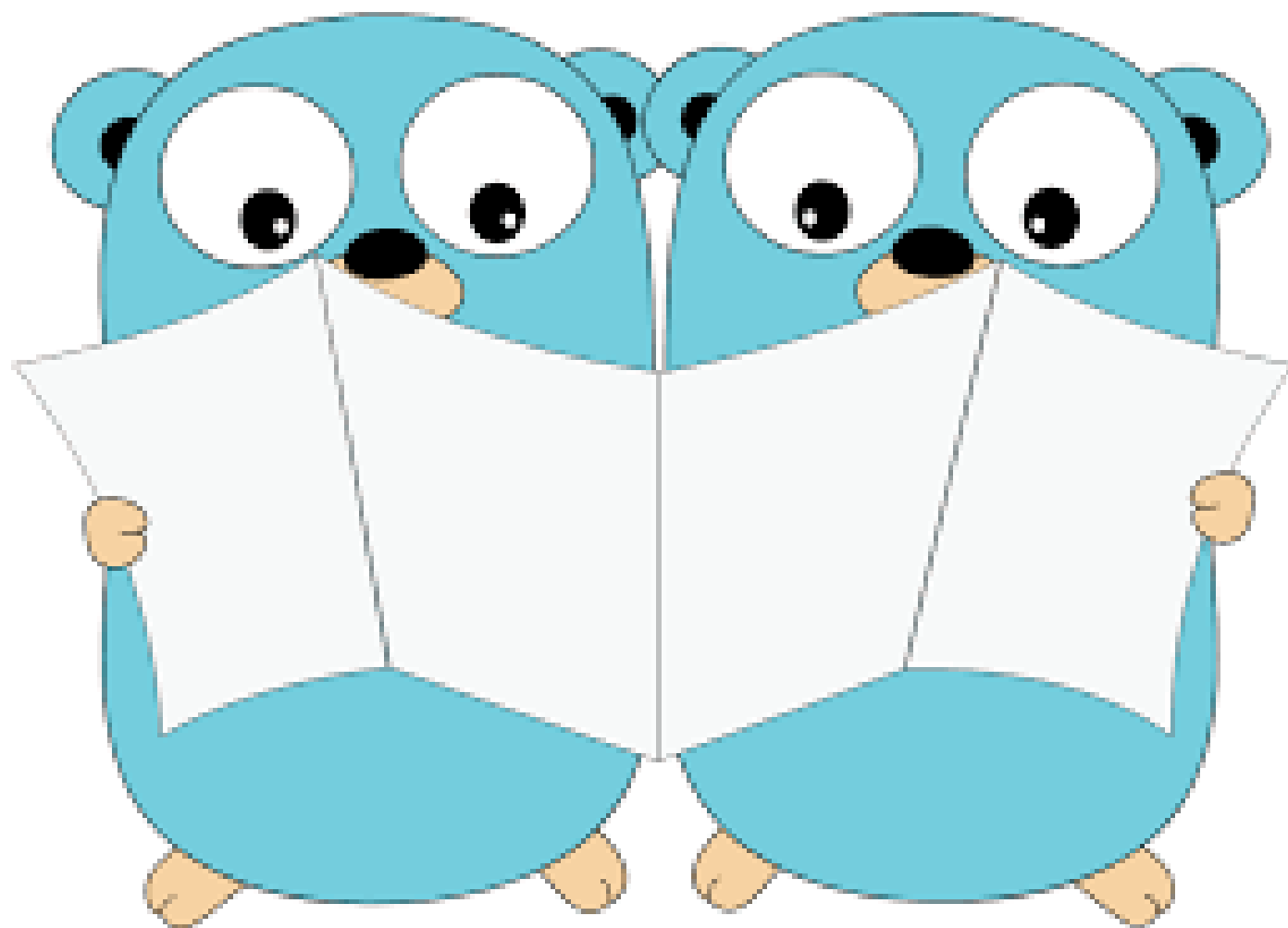


Building Ecommerce $\equiv$ GO using Microservices



Contents



1 What is microservice

2 What is Domain Driven Design

3 Interface layer

4 Application layer

5 Domain layer

6 Infrastructure layer

7 Project

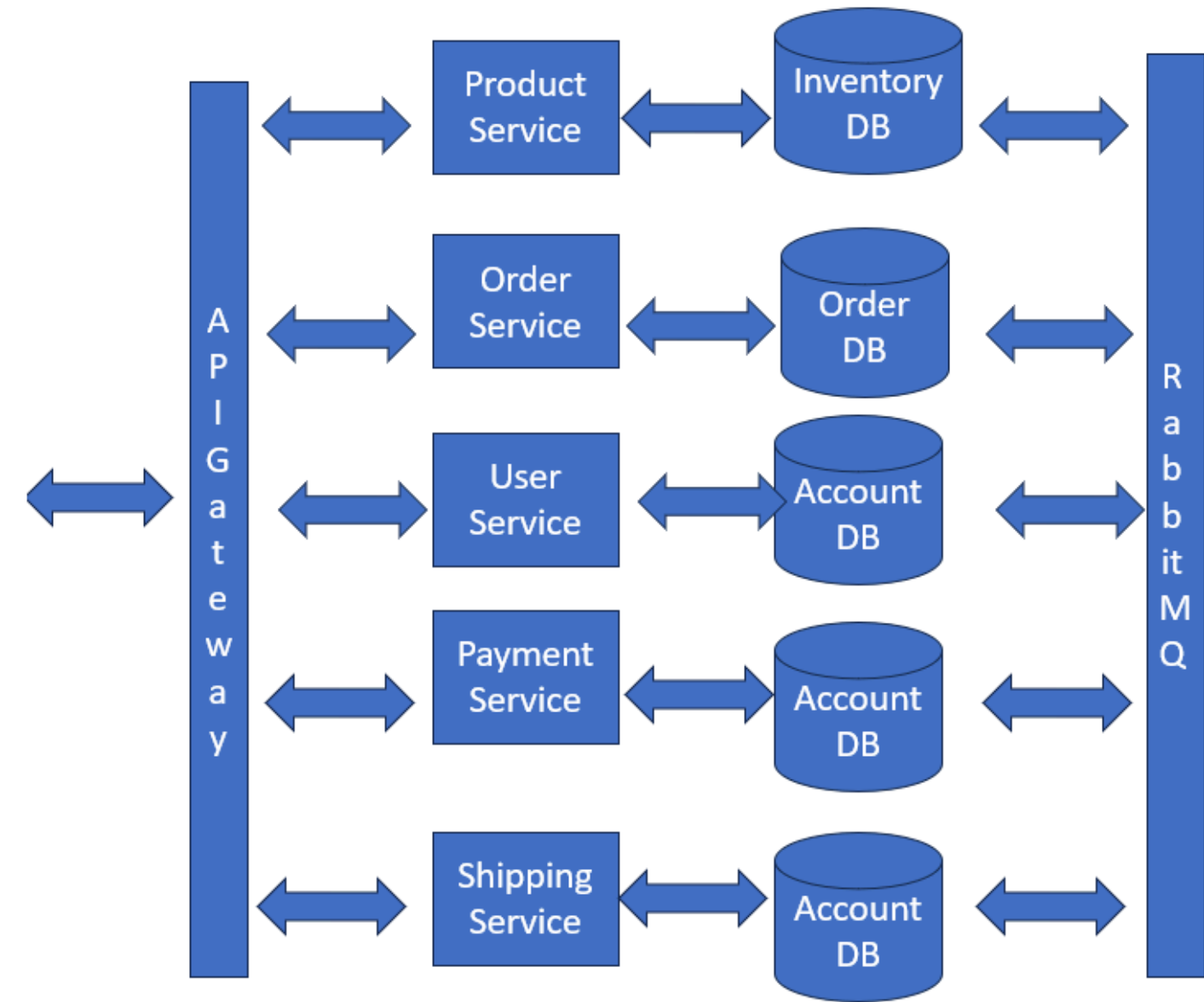
microservice architecture

A microservice is an independent component of a business logic. It is mostly a breakdown of a monolith application into individual components that can scale and make the service fault tolerant.

There are many advantages of microservices and they cater to different business needs as per design architecture.

They can be :-

- Event driven
- Domain driven
- with webworker



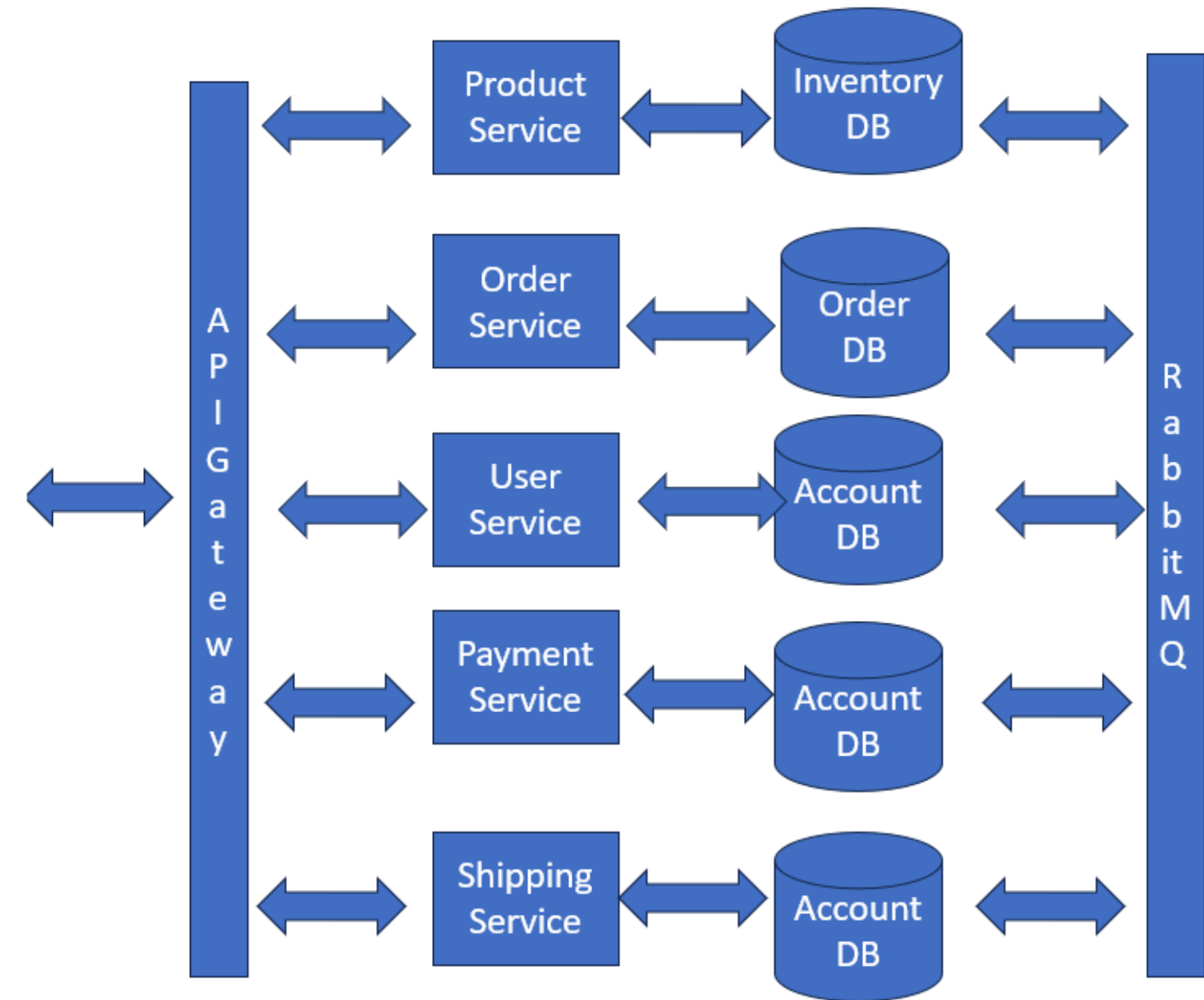
key services

A microservice is an independent component of a business logic. It is mostly a breakdown of a monolith application into individual components that can scale and make the service fault tolerant.

There are many advantages of microservices and they cater to different business needs as per design architecture.

They can be :-

- Event driven
- Domain driven
- with webworker



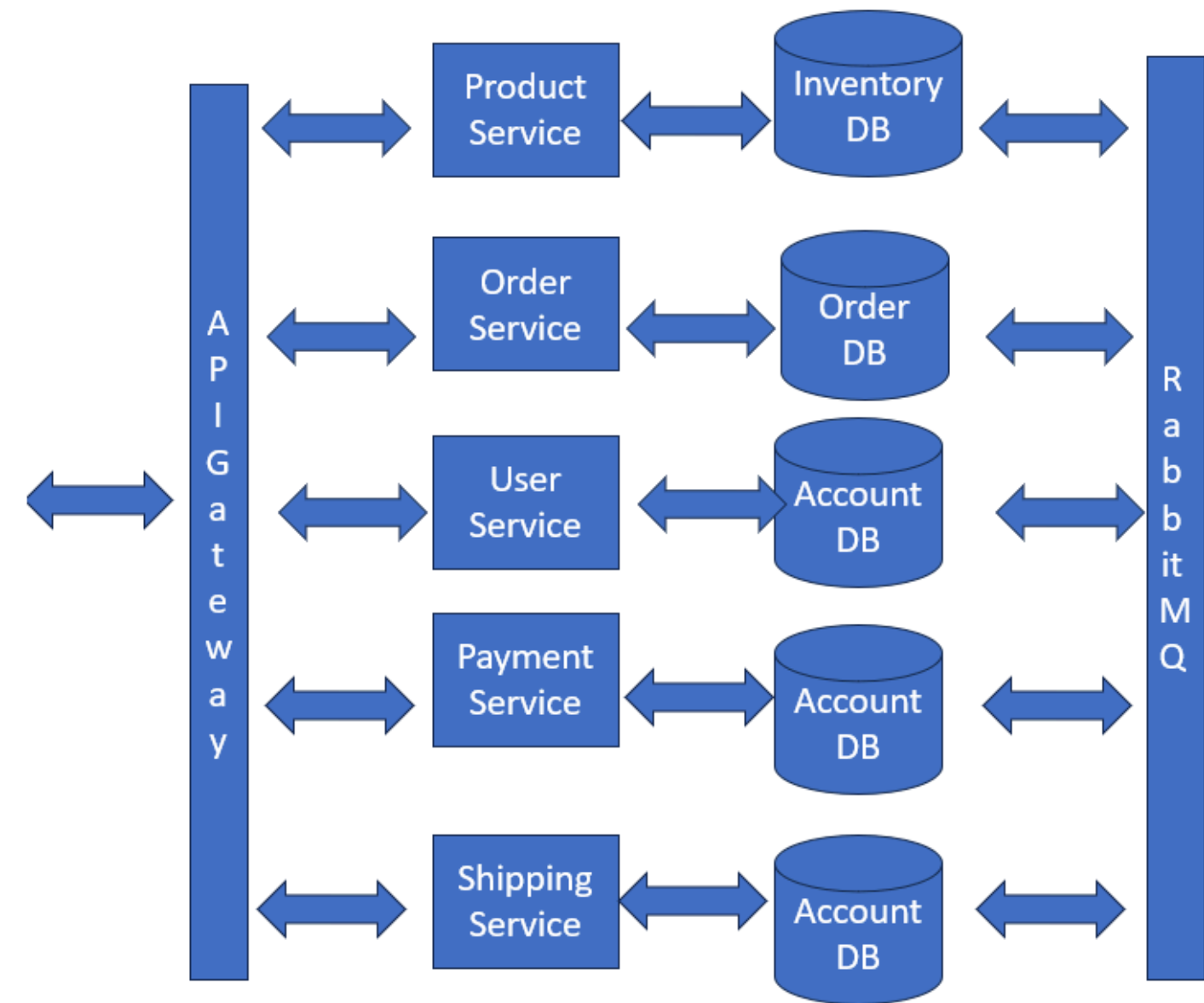
api gateway

A microservice is an independent component of a business logic. mostly is a breakdown of a monolith application into individual components that can scale and make the service fault tolerant

There are many advantages of microservices and they cater to different business needs as per design an architecture.

They can be :-

- Event driven
- Domain driven
- with webworker



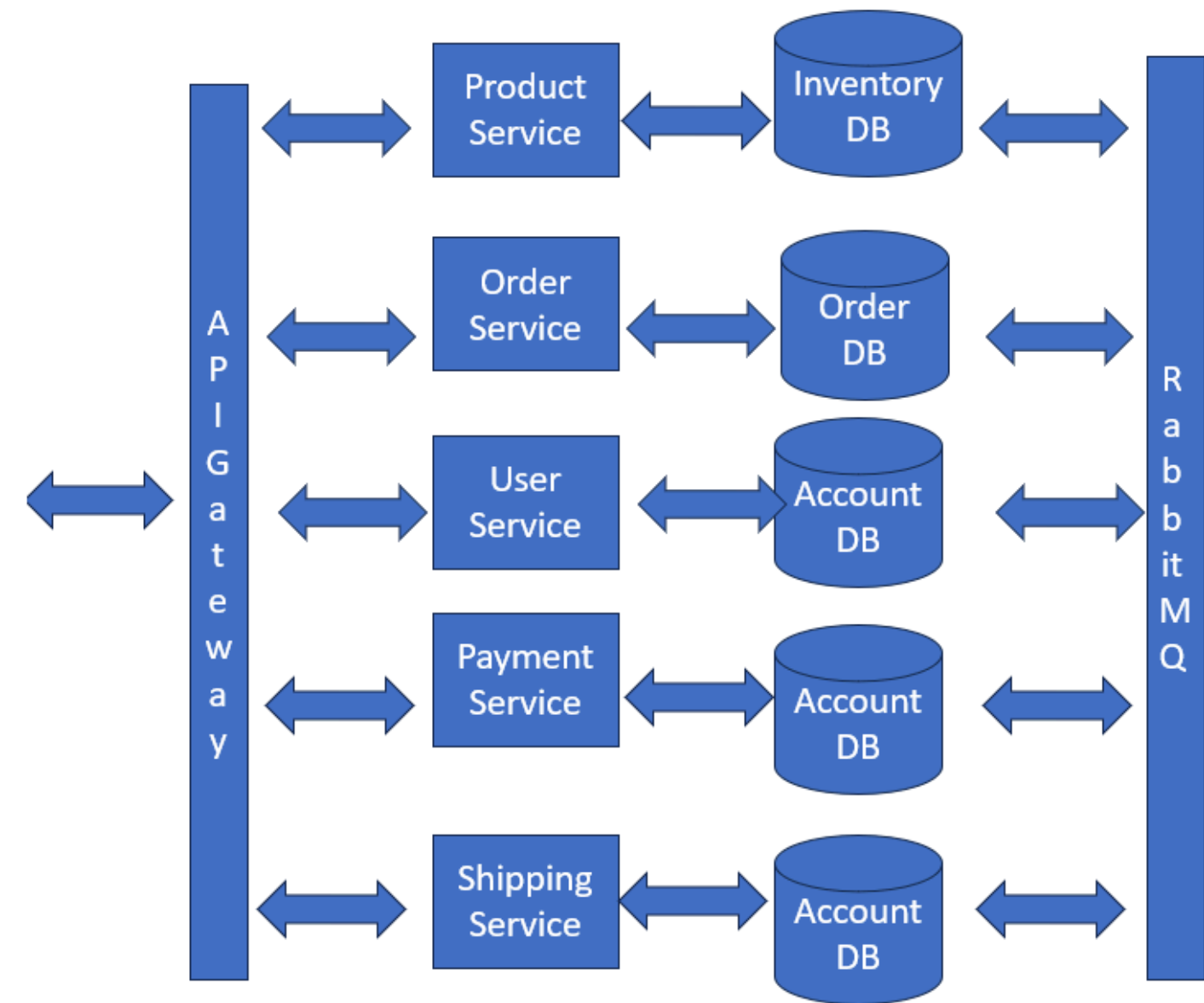
inter service communication

A microservice is an independent component of a business logic. It is mostly a breakdown of a monolith application into individual components that can scale and make the service fault tolerant.

There are many advantages of microservices and they cater to different business needs as per design architecture.

They can be :-

- Event driven
- Domain driven
- with webworker



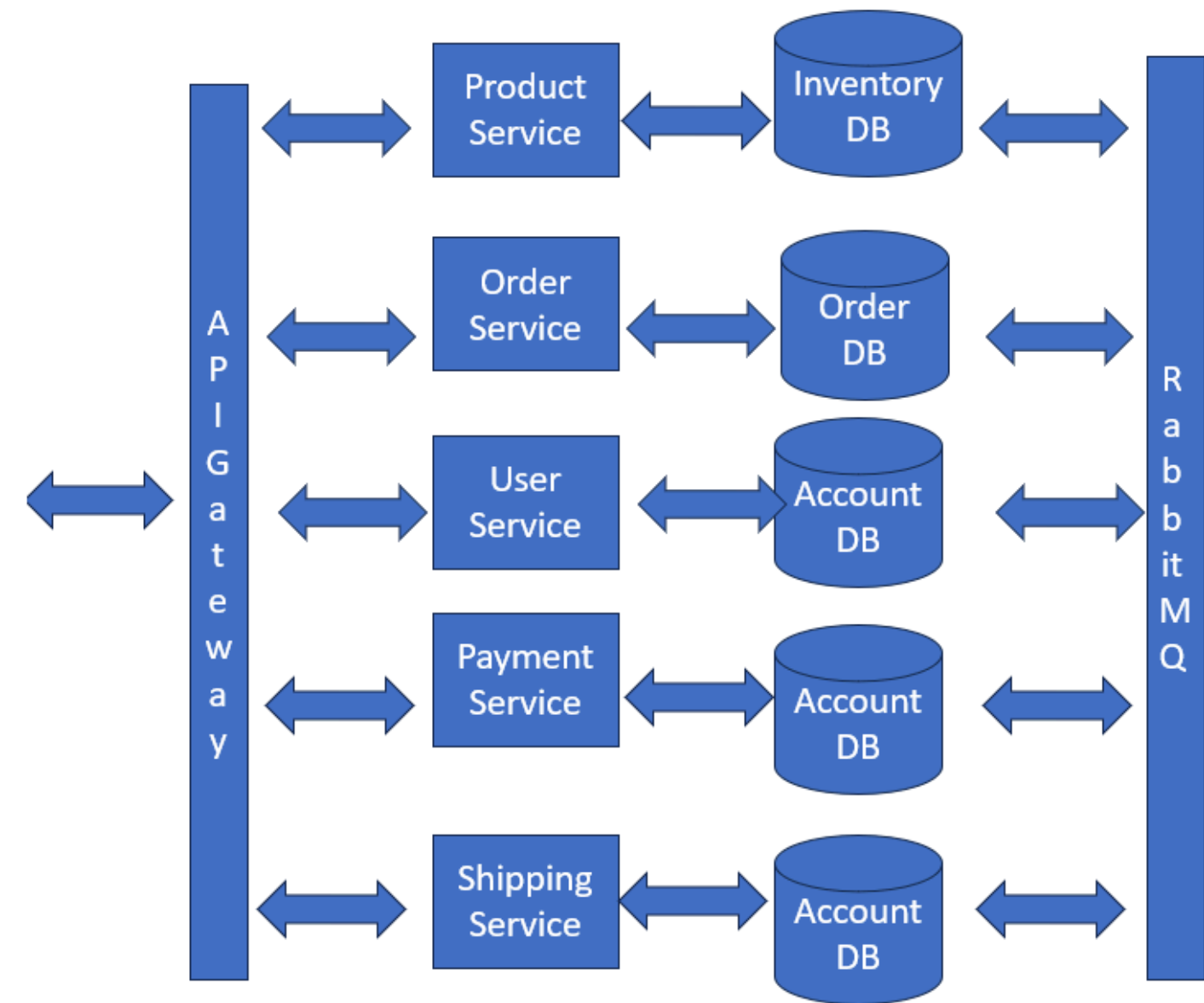
service discovery and configs

A microservice is a independent compoent of a business logic. mostly is a breakdown of a monolith application into individual componetns that can scale and make the service fault tolerant

There are many advantages of microservces and they cater to different business needs as per design an architecture.

They can be :-

- Event driven
- Domain driven
- with webworker



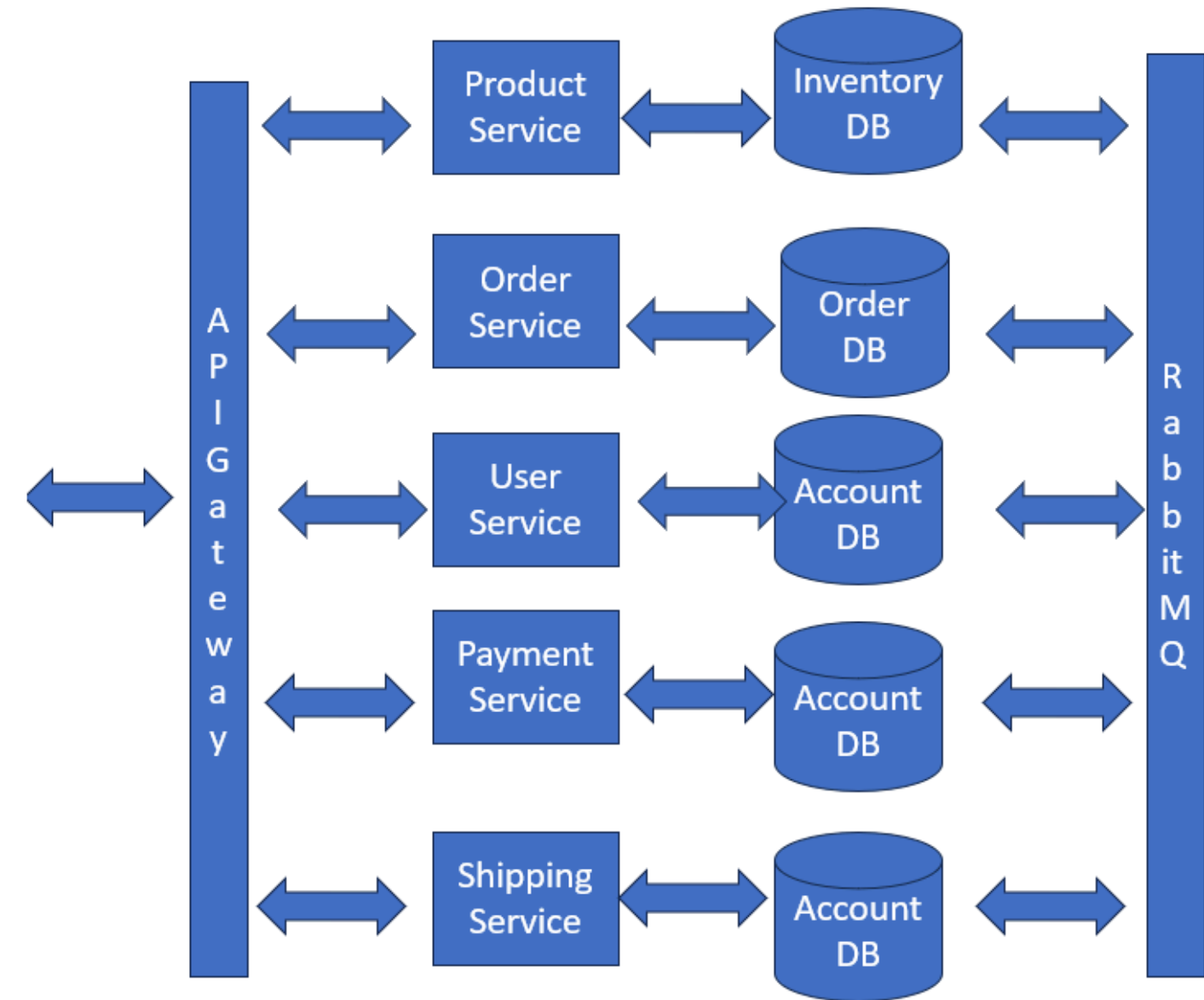
What is a Microservice

A microservice is a independent compoent of a business logic. mostly is a breakdown of a monolith application into individual componetns that can scale and make the service fault tolerant

There are many advantages of microservces and they cater to different business needs as per design an architecture.

They can be :-

- Event driven
- Domain driven
- with webworker



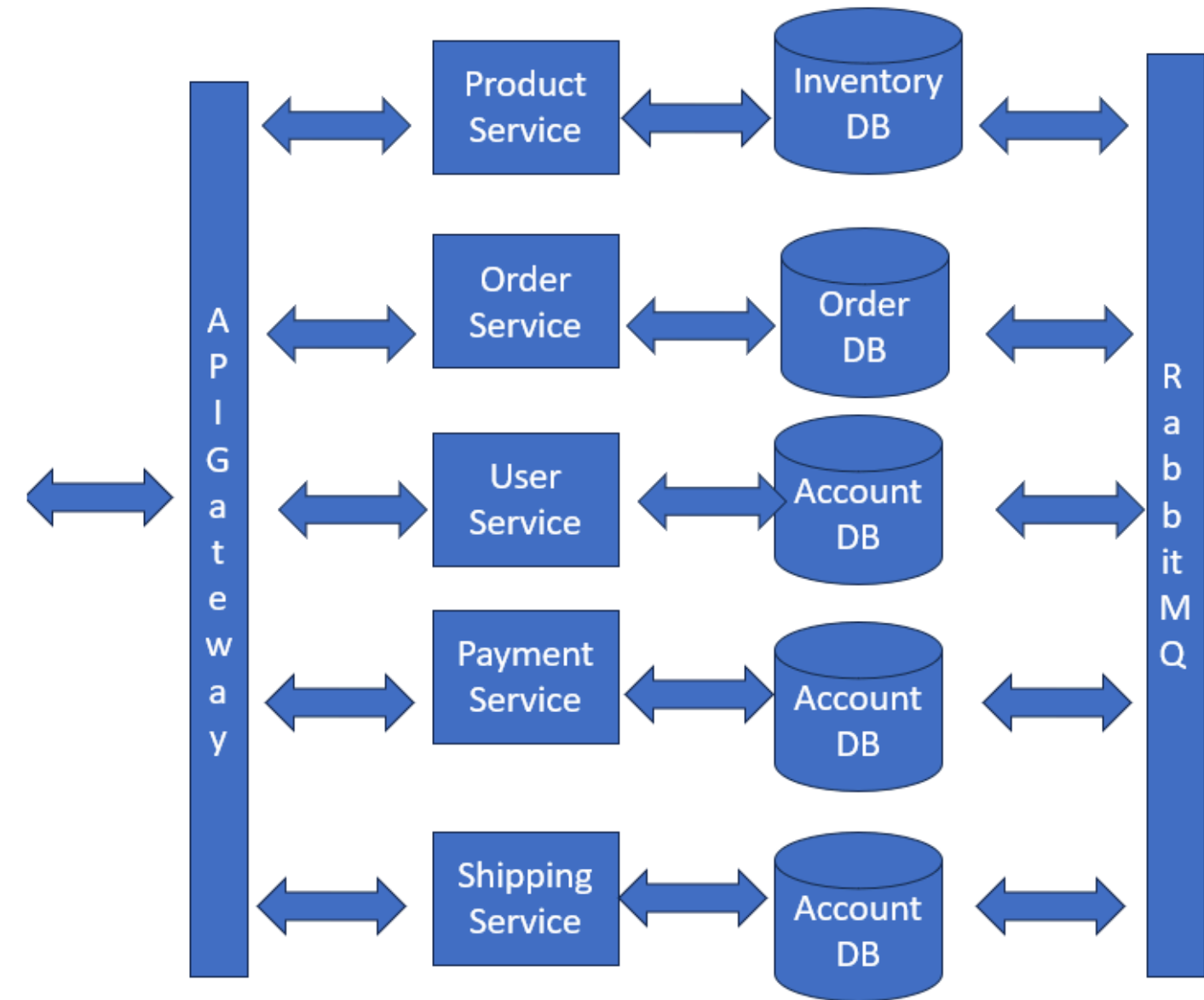
What is a Microservice

A microservice is a independent compoent of a business logic. mostly is a breakdown of a monolith application into individual componetns that can scale and make the service fault tolerant

There are many advantages of microservces and they cater to different business needs as per design an architecture.

They can be :-

- Event driven
- Domain driven
- with webworker



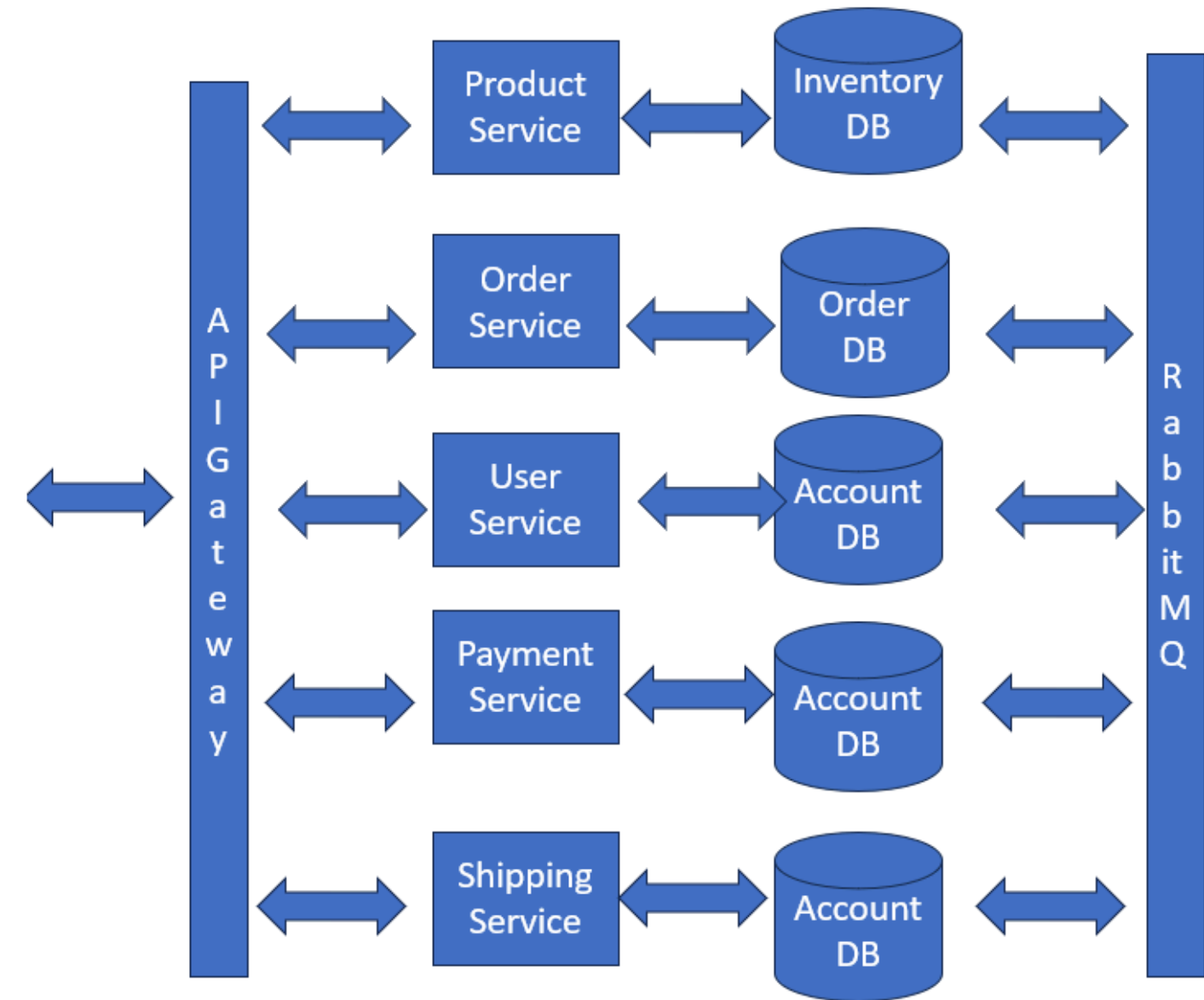
What is a Microservice

A microservice is an independent component of a business logic. It is mostly a breakdown of a monolith application into individual components that can scale and make the service fault tolerant.

There are many advantages of microservices and they cater to different business needs as per design architecture.

They can be :-

- Event driven
- Domain driven
- with webworker



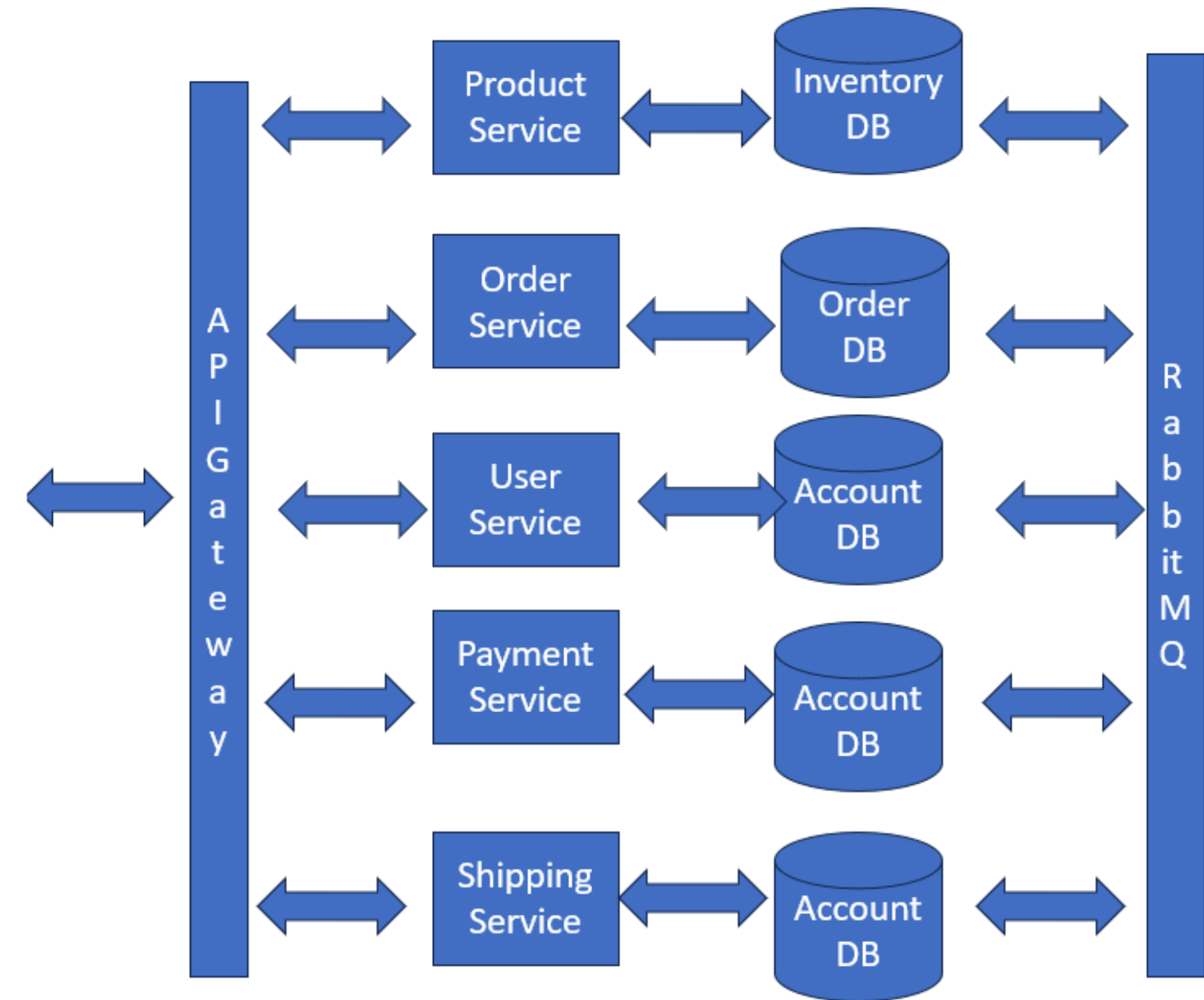
What is a Microservice

A microservice is a independent compoent of a business logic. mostly is a breakdown of a monolith application into individual componetns that can scale and make the service fault tolerant

There are many advantages of microservces and they cater to different business needs as per design an architecture.

They can be :-

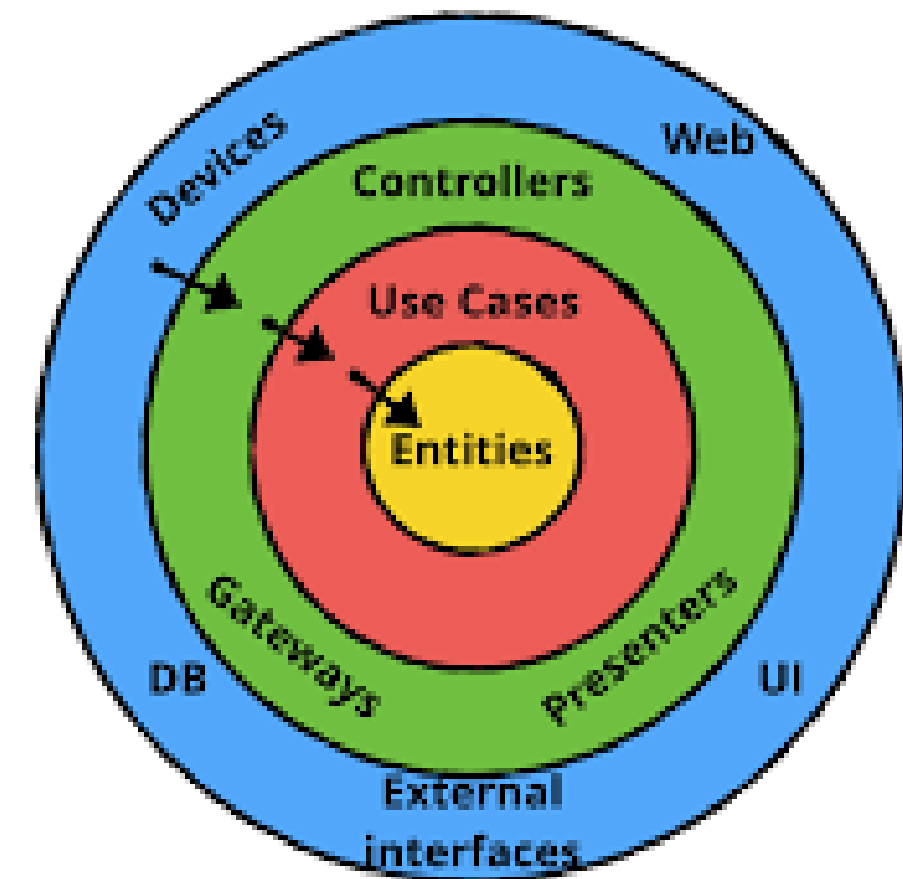
- Event driven
- Domain driven
- with webworker



What is Domain Driven Design

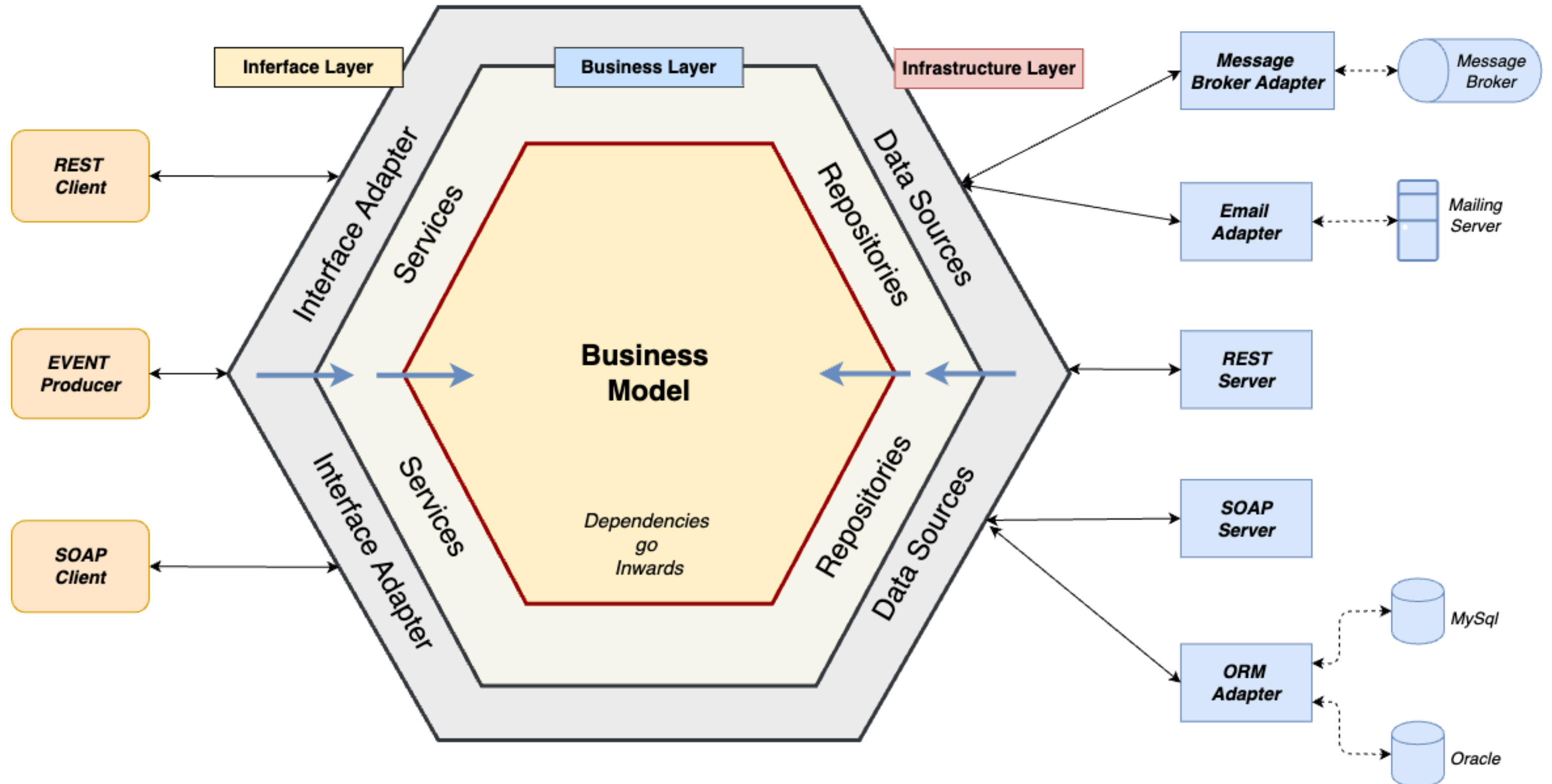
Domain Driven Design is a software design pattern that separates business logic from language infrastructure and other technical components. It has business logic domain at its core and all other things that change are wrapped like onion onto it.

It is important as the core business logic never changes but the outlying technologies change but do not affect the business logic. It is like a port and adapter pattern where technologies like databases or message queues can be replaced easily without affecting anything.



What is Domain Driven Design

Hexagonal Architecture

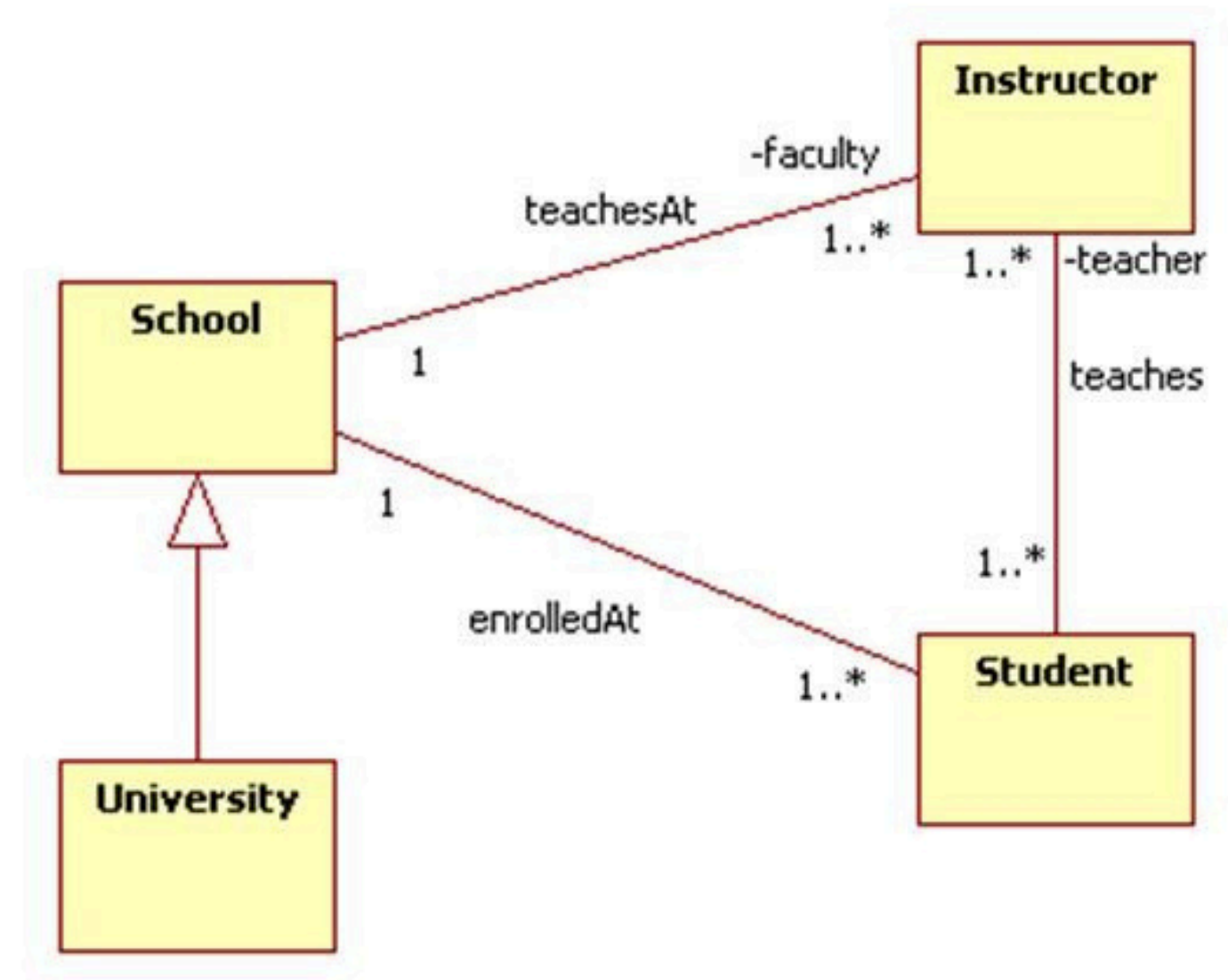


Domain Layer

The main business layer of the application that contains business models , methods, events and their working.

These are independent of any database or message queue or router and they mostly dont change.

they have classes or structs that defines objects or models and their methods and their interface signatures .

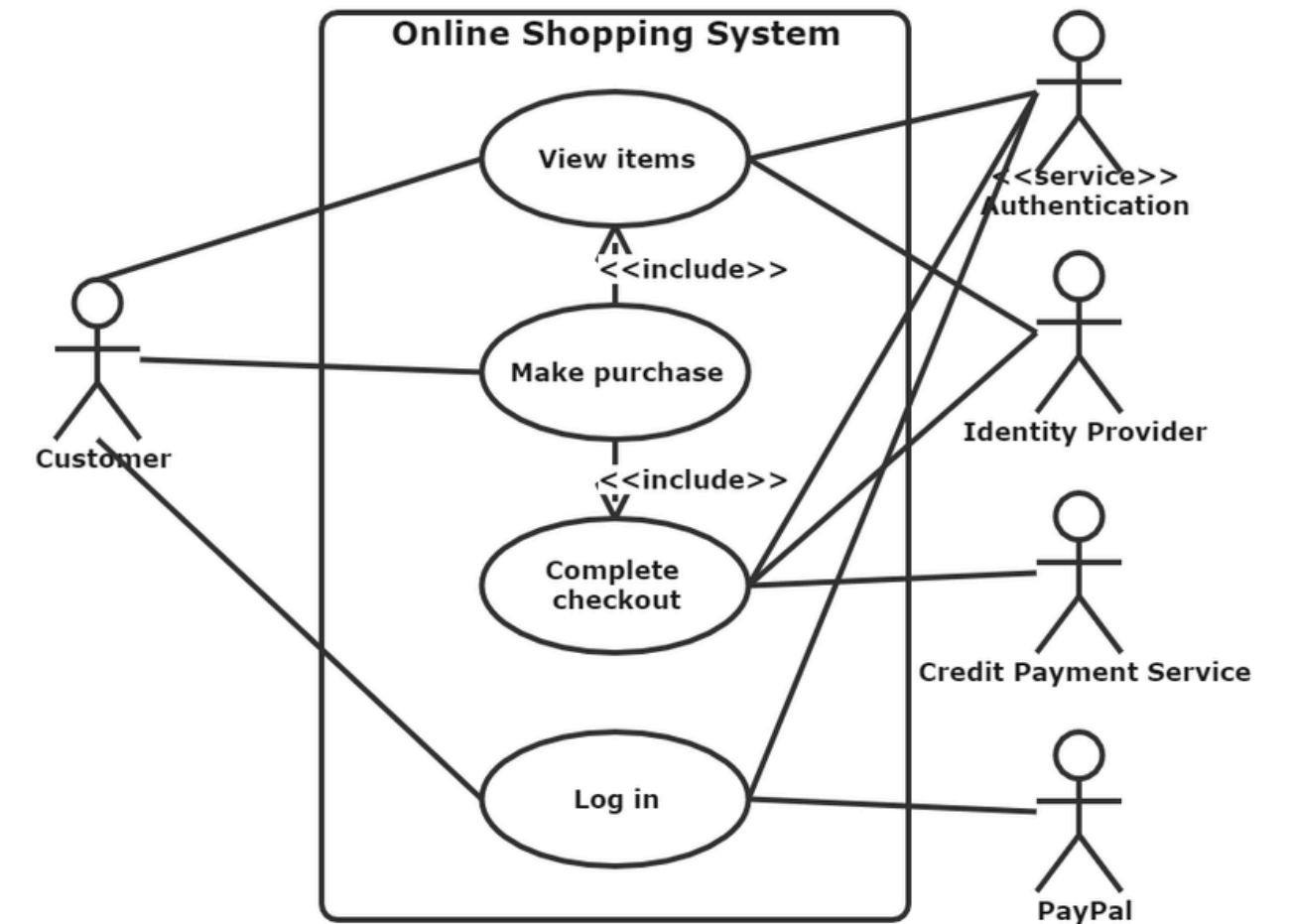


Application Layer

Application Layer is the layer that connects interface layer to the domain layer.

here there are many usecases of business present that may use more that one domain method for its working. along with that it is responsible for authentication and transaction processing . sending notifications or triggering events.

when interface later recieves a resuquest it sends to appropriate usecase method in application layer that may contain multiple domain methods and triggers events accordingly.



Interface Layer

Interface layer is the outer layer from where the application is accessed .

it can be a cli , a rest api or a grpc endpoint. even all three in one application .

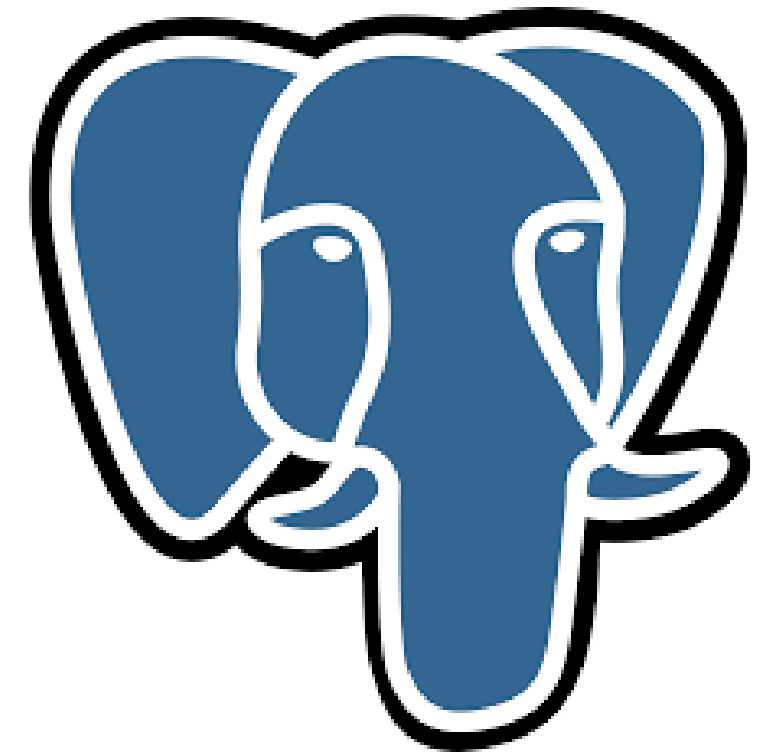
This takes the appropriate input and sends to the application layer usecase for further processing and sends response back.



Infrastructure Layer

Infrastructure layer is where all interfaces are implemented concrete. It has all the technologies in it such as database, message queue, external apis and other relevant external things that are required by the application.

It should implements domain interface methods to work and this infra related technologies are injected into the repository or other related objects in the main setup. this process is called wiring.



Project

Github Link

<https://github.com/subpxl/ecommerce-microservices>

```
1  product-service/
2  |
3  |— cmd/
4  |   └─ main.go           # Entry point
5  |
6  |— internal/
7  |   |— application/
8  |   |   └─ service.go    # Application
9  |   |   └─ dto.go        # Command/Query
10 |
11 |   |— domain/
12 |   |   └─ product.go     # Entity & bo
13 |   |   └─ repository.go  # Domain inte
14 |
15 |   |— infrastructure/
16 |   |   └─ repository/
17 |   |       └─ mysql.go    # MySQL imple
18 |
19 |   └─ interfaces/
20 |       └─ http/
21 |           └─ handler.go  # REST handle
22 |           └─ router.go   # HTTP routi
23 |
24 └─ go.mod / go.sum
```

[Github.com/subpxl](https://github.com/subpxl)

Shubham Panchal

